Chapter route

How can enforceable structures limit a subject's authority over objects and preserve provable consistency after revocation?

Prerequisites are mode bits, access control, graphs, and matrices.

- Distinguish subjects, objects, domains, and rights.
- Compare ACLs and capabilities.
- Analyze revocation of existing handles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Who may perform which operation on an object?

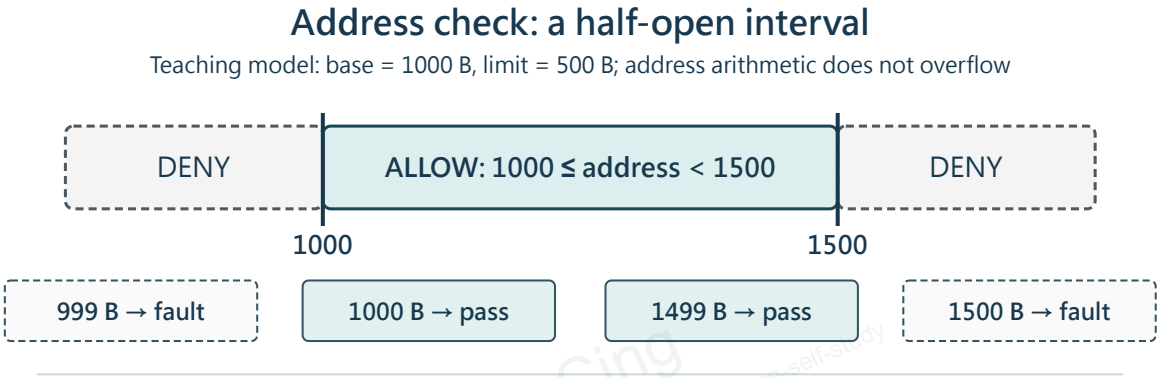
A **subject** is an actor issuing requests, such as a process. Its current **protection domain** is the set of object-right pairs it can exercise; an authorized domain switch can change that set. An object is a resource, and a right denotes an operation. Subject and domain are distinct concepts.

A policy may let assistants read submissions and only the instructor change grades. ACLs, capabilities, and protected system calls are mechanisms that enforce it. Complete mediation requires effective authorization for each use. Checking an ACL at open and a protected handle's rights on each read/write also requires a defined revocation policy for issued handles.

A reference monitor must resist bypass, protect its own state, and be small enough to inspect. Hardware modes and address-space isolation support protection but do not prove application policy correct. Security also addresses authentication, attacks, faults, auditing, and trust; an obscure pathname is not access control.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Base and limit: enforcing an OS policy



Mechanism: compare base ≤ address < base + limit; otherwise raise a fault.

Policy: the OS decides the permitted range and securely sets base and limit.

This byte-addressed example checks a physical address against `[base, base + limit)` and assumes the addition cannot overflow. The OS chooses the permitted range and protects its configuration; hardware performs the comparison and raises a fault for an out-of-range address. Actual hardware may check a relative address before relocation, so this diagram is a model, not a universal implementation.

Least privilege includes scope and time

A backup job needs read access to /course/submissions and write access to its backup target, not grade changes or module loading. Running the whole parser with full authority lets a malicious input exploit every permission. A low-privilege parser plus a small authorized service limits exposure, provided that service validates parameters and scope.

Granted rights R(t) must include currently necessary N(t), while minimizing the excess $R(t) \setminus N(t)$. Counting permissions alone is inadequate. Reading every student's personal data can be riskier than writing one temporary file. Scope, duration, and delegation affect the damage bound.

Fail-safe defaults deny absent explicit authorization. Audit subject, object, operation, decision, time, and correlation ID; logging does not itself block an attack. Separation of duty can require two roles to publish grades, but fails if one person can freely activate both roles.

A domain is a set of rights

Let D={⟨F1,{read}⟩,⟨F2,{read, write}⟩}. Two processes of one user can have different domains; a protected call within one process can switch domains. User/kernel modes alone do not separate user-mode processes, which also need address spaces and authorization.

A static domain unions phase requirements, such as reading source during compilation and writing binaries during linking, exposing both rights throughout. A dynamic domain narrows each phase, but its switches must be authorized and preserve/restore caller context correctly while validating arguments.

A traditional UNIX setuid executable can change effective UID to its owner on successful exec under specified conditions. It does not necessarily change real UID or act like an ordinary function call. With real UID1001 and owner0, an allowed setuid exec can produce euid0 while retaining real UID1001. The program must validate input/environment and drop privileges correctly. OS-specific suppression conditions still apply.

Multics rings and controlled gates

In the historical abstraction, ring 0 is more privileged. An access bracket (b1, b2) and gate limit b3≥b2 allow direct execution from ring i in [b1, b2]. For i>b2, entry into a more privileged ring requires i≤b3 and a designated gate. For i<b1, transfer toward lower privilege must not let parameter references expose memory unavailable to that callee.

With bracket[1,3], limit5, and gate entryA, ring 2 may use an ordinary entry and remain ring 2. Ring4 may enter ring 3 through entryA, but entryB is denied. Ring6 is denied even at entryA. Return restores caller ring/stack. Full Multics also has read/write brackets beyond this model.

Nested privilege cannot express incomparable requirements. If one domain needs only F and another only G, forcing either to contain the other grants an unnecessary object. A general access matrix supports this separation.

The initial matrix

Access matrix and domain-switch reachability

domain	F	G	P	D2	D3
D1	r*	∅	∅	switch	∅
D2	w	r	∅	∅	switch
D3	∅	∅	print	∅	∅



Switch path: D1 → D2 → D3

Current rows are not unioned

r* = read plus same-object delegation; ∅ = no rights. Switch paths do not authorize arbitrary entry points.

A[D, O] is a set of operations; a blank is ∅. The process uses its current row. D1 may read F, not write it; D2 may write F and switch to D3; D3 may print P. Because D1 can switch to D2 then D3, its initial row does not prove it can never print.

Switch is a right over a domain object. Switching selects another row rather than unioning both. A controlled service also needs entry/parameter restrictions against jumping into arbitrary privileged code.

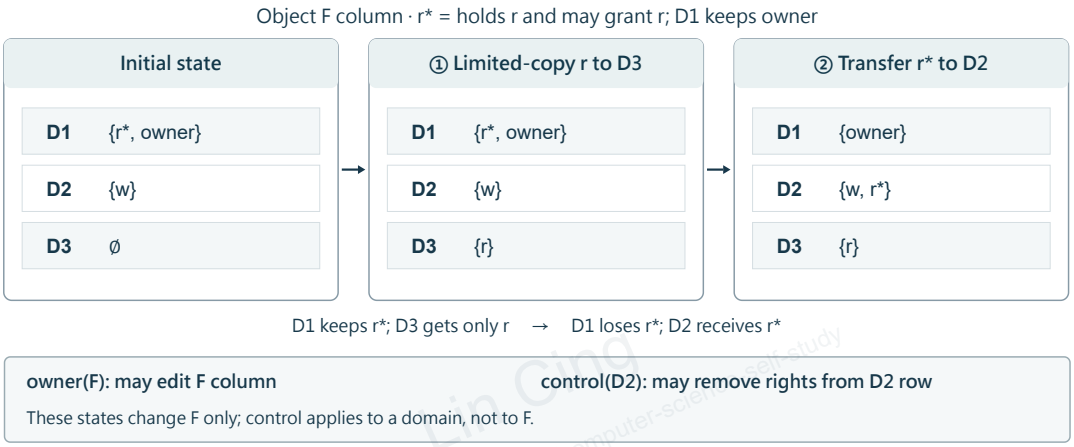
Copy, transfer, limited copy, owner, and control

Right `r*` grants r and permits its propagation within the **same object column**. F:`r*` does not authorize G:r. Ordinary copy retains the source; this model's transfer removes `r*` from the source and gives it to the target; limited copy grants r without propagation authority.

Independent initial state. This example does not continue the preceding matrix: A[D1,F]={`r*`, owner}, A[D2,F]={w}, A[D3,F]=∅. After D1 limited-copies r to D3, D3={r} and cannot delegate that right using this grant. As F's owner, D1 may add or remove specified rights in F's column, including D2's w. Ownership of F does not imply ownership of every object.

Control applies to a domain object. With control in A[D1,D2], D1 may remove rights from D2's row, but not freely add them. Removing D2's switch to D3 breaks the path D1→D2→D3; check separately for alternative paths.

Track copying and transfer in one column



This is an independent initial state; control and owner are distinct rights.

Rights propagation does not prevent information flow

D3 without copy(F) can still read F and write each byte to G if it holds those data rights. Copy rights govern delegation, not information copying. Revoking F:read also cannot ordinarily recall bytes already read into user memory.

Consider a restricted system where the only grant rule attenuates existing source rights in the same column, with no ownership or other creation of rights. Every right in every successor state originates in an initial right for that same object. The initial state satisfies the claim. Inductively, each new r has a source r, whose provenance reaches the initial state. This proves the claim for arbitrary finite transition sequences. It neither prevents data leakage nor applies when an owner can create rights freely.

Sparse representations

Representation	Index	Convenient query	Protection/update requirement
Global triples (D, O, R)	Global or dual indexes	Any pair	Protect table and avoid a full scan per use
ACL(O)=[(D, R)]	Object column	Who may use O?	Central deletion; explicit matching/default rules
Capability list(D)	Domain row	What authority does D hold?	Unforgeable references; controlled transfer/attenuation
Locks/keys	Object locks and domain keys	Is a key accepted?	Protect lists; change keys to revoke

A capability is a protected object reference with rights. A numeric fd indexes the current process's table; guessing another process's number does not acquire its capability. Kernel tables, tagged hardware, or another protected representation can ensure unforgeability. Capabilities need not universally be absent from user address spaces.

Convert columns to rows without changing rights

F has ACL[D1:r, D2:rw]; G has[D2:r, D3:w]. Corresponding lists are D1=[F:r], D2=[F:rw, G:r], D3=[G:w]. Both represent one matrix at one instant, but delegation/revocation costs differ. Hash-indexed pair queries can have expected O(1) cost; a linear list of length k costs O(k). The representation's name does not imply constant time.

A hybrid checks ACL at open and issues a kernel handle{F, read}; write later fails its mode check. If the ACL subsequently removes read, continued use depends on revocation policy. UNIX generally does not promise immediate invalidation of every open fd. Per-use generation checks can change this contract, with mappings/in-flight operations handled separately.

This chapter's simplified ACL uses allow entries with union and default denial. Deny ACEs, ordering, and group rules in real systems need their own algorithm. Do not substitute this model for Windows DACL evaluation.

Users, roles, and task permissions

Solaris10 separates roles, rights profiles, authorizations, and process privileges. A role groups administrative tasks; a profile can specify commands/execution attributes; an application can check authorizations; the kernel checks privileges. Role authority is not merely a UID, and an authorization string is not itself a kernel privilege.

In the original teaching model, U assumes BackupRole and executes BackupCmd with only source-read and target-write rights, unable to change GradeDB. V assumes GradeRole but cannot approve its own publication. Role activation still needs authentication/auditing. User-to-role assignment alone does not allow every request.

Formally $UA \subseteq Users \times Roles$, $PA \subseteq Roles \times Permissions$, and $activeRoles(session) \subseteq assignedRoles(U)$. A request passes iff some active role contains the required permission, subject to activation constraints such as exclusion. This simplified RBAC equation is not all Solaris privilege-set computation.

Four independent revocation dimensions

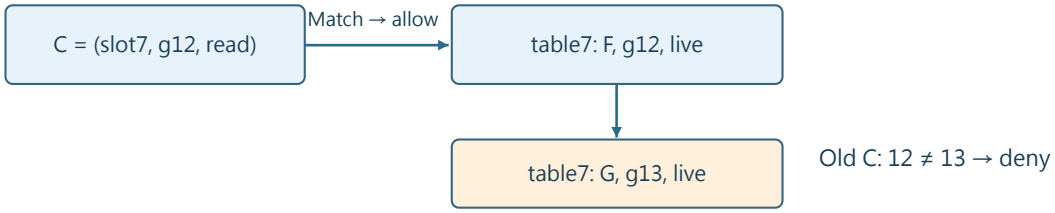
Immediate/delayed specifies when revocation takes effect; selective/general specifies holders; partial/total specifies operations; temporary/permanent specifies future regranting. Even immediate revocation needs a boundary: denying later operations does not automatically cancel in-flight I/O or erase previously read data.

An ACL centralizes removal, but issued handles or caches may delay its effect. Distributed capabilities can use reacquisition/leases, back-pointers, indirection, or keys/versions. Back-pointers must track all copies; indirection centralizes checks at a revocable slot. Changing a shared master key revokes all capabilities using it; selective revocation needs separate key groups or proxies.

Lease example. Use a consistent trusted time base. A grant begins at t=0 and expires at t=10 seconds; every new operation checks expiry. At t=3 the authority revokes the grant and refuses renewal. Even if notification is delayed, operations starting at $t \geq 10$ are denied by expiry, so the delay is at most 7 seconds. Unlimited caching, use after expiry, or renewal after revocation invalidates this bound. A separate cancellation contract governs operations already in flight.

Indirection and generation numbers

Reusing a slot: index 7 alone is insufficient



Change generation on reuse. Synchronize checking and retaining the object during resolution.

Capability C={slot7, generation12, rights r}; table[7]={F, g12, live}. Resolve checks liveness, generation equality, and rights, then obtains a stable object reference under the same synchronization. Revocation clears live; reuse for G increments generation to13. Old C=(7,12) fails against(7,13).

Assume generations are never reused while old tokens may exist, and revocation/reallocation cannot interleave between checking and retaining the object. Then every accepted C identifies the object version for which it was issued. Finite wrapping counters with indefinitely retained old tokens eventually permit ABA. Prevent reuse or prove old-token quiescence before wrapping. Checking only slot7 demonstrably grants G to F's old capability.

Hydra and restricted amplification

Historical Hydra supports auxiliary rights for user-defined types. A client capability for Account A contains deposit, not raw representation read/write. Calling trusted deposit(A,10) checks type and operation rights; the protected procedure temporarily obtains representation access, changing balance40→50. The client still holds only deposit afterward.

Only approved procedures, types, and rules amplify authority. Arbitrary routines that turn read-only arguments into writable ones would violate callers' protection. Mutually distrustful subsystems must constrain amplifiable rights, retained references, and parameter scope; encapsulation alone is insufficient.

Cambridge CAP data and software capabilities

Machine/microcode enforces data-capability segment read/write/execute rights. Protected procedures interpret software capabilities' custom meanings; the lower mechanism prevents forgery/modification. Seal/unseal-style authority limits representation inspection to the matching type/environment.

Both CAP and Hydra give clients abstract operations while implementations require lower-level access through a protected typed boundary. Their rights interpretation/support libraries differ. Running arbitrary user code does not itself prove safety. A Bank procedure that corrupts balance still breaks Bank semantics; isolation limits damage to its environment without proving business invariants. Account values here are an original model, not executable Hydra/CAP APIs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

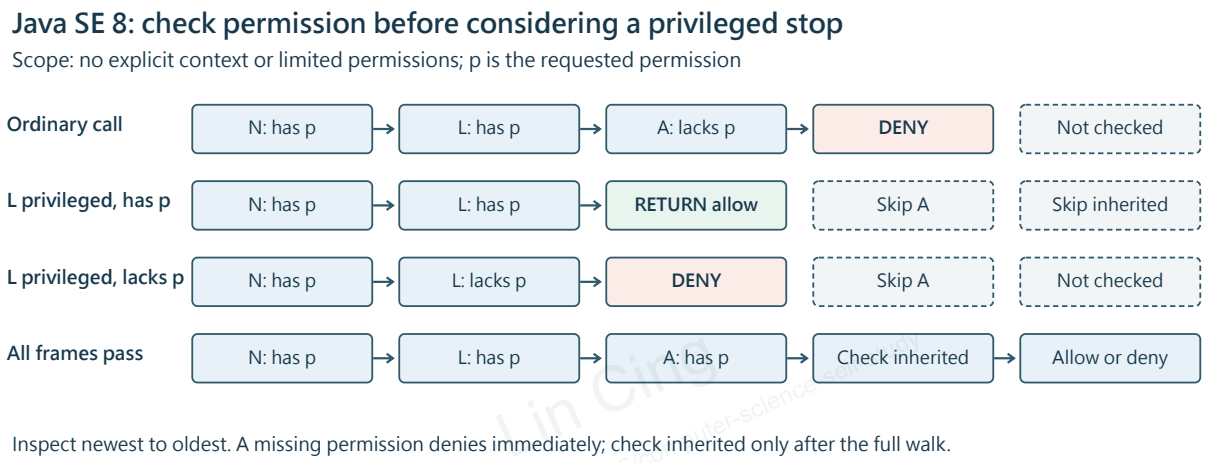
Compile-time, load-time, and runtime guarantees

Types/modules/interfaces declare access boundaries. Compilers reject statically invalid operations; runtimes check dynamic indices, types, and capabilities. If an attacker can alter machine code, forge references, or bypass checks through native entries, source-level private fields alone cannot isolate it.

Type safety constrains memory/type behavior, not business authorization. A valid Account reference can still be misused to allow withdrawals or leak information. Kernel isolation separates processes; language isolation can separate classes/objects within a process, adding runtime, loader, and native interfaces to the trusted computing base.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical Java SE 8 stack inspection



In the Java SE 8 model, doPrivileged has neither explicit context nor limited permissions. Stack N→L→A is newest first; N and L have permission p, A lacks it. Ordinary inspection denies at A. A privileged L permits immediately after N and L pass, skipping A and inherited context; if L lacks p, it still denies.

Only when inspection reaches the end without such an early return and all frames pass does it check the thread’ s inherited context. Other overloads have additional rules. This is a historical security model, not a modern sandbox recommendation.

Confused deputies and validated targets

Loader can connect to a fixed proxy for a low-privilege applet within a controlled privileged block. Passing arbitrary applet addr directly into that block lets the caller use Loader to scan an internal network. Restrict validated destination/port, operation, and returned data; do not execute an entire untrusted callback with elevated authority.

Inspection maintains that every checked frame holds p. A privileged stopping frame must also hold p. Thus this authorization never exceeds that boundary caller's existing p. Correct object/argument selection still depends on the caller; the stack theorem does not prove the service policy.

Equivalence from matrix to implementation

The matrix states current permission; ACLs/capabilities index it; dynamic operations change it. Preserve object identity, right types, and domain context with unforgeable, non-bypassable checks. Cached decisions require explicit revocation semantics.

Owner changes a column; control deletes from a domain row; switch changes current domain; copy/transfer/limited copy change same-object grants. Their composition changes reachable domains, so one static row is not a whole-program authority bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three distinct safety claims

Authorization safety asks whether each operation has a right. State safety asks whether revocation/reuse/races identify the wrong object. Information/business safety asks whether legal reads/writes or privileged services still leak data or violate policy. Each needs different evidence.

State allowed transitions before proving a general claim. Attenuation induction excludes arbitrary owner grants; generation proofs exclude finite wrap with everlasting old tokens; Java privilege does not excuse parameter validation. Exercises use full columns, versions, and stacks to expose these conditions.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 14. Oracle: Java SE 8 AccessController. Oracle: Solaris 10 RBAC.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study